

Objectif

Savoir réaliser et mettre en place une architecture REST complète en Java EE. Ce projet couvre la manipulation de ressources complexes, l'intégration de logique métier, la gestion fine des codes de retour HTTP, le support multi-format (JSON/XML) et la sécurisation par token.

Contexte : Le projet EcoDrop

L'entreprise `EcoDrop` souhaite mettre à disposition des entreprises tierces et des citoyens une API de gestion de collecte de déchets spécifiques (batteries, textiles, électronique). L'application doit permettre de gérer les points de collecte, les types de déchets acceptés et les dépôts effectués par les utilisateurs.

L'entreprise compte sur vous pour développer son service REST offrant différentes fonctionnalités à ses usagers.

1 Partie A

1.1 Le Catalogue de Déchets

On s'intéresse tout d'abord à la table `WasteType(id, nom, pointsPerKilo)` contenant les types de déchets traités par l'entreprise

- Créer le DAO et éventuellement le DTO pour les types de déchets.
- GET `/waste-types` : Liste tous les types de déchets disponibles. Ce endpoint doit supporter `application/json` et `application/xml`.
- GET `/waste-types/id` : Détails d'un type spécifique. Renvoie 404 si l'ID n'existe pas.
- POST `/waste-types` : Ajoute un nouveau type.
- PUT `/waste-types/id` : Mise à jour complète d'un type.
- DELETE `/waste-types/id` : Supprime un type. Renvoie 409 Conflict si le type est utilisé dans la table des dépôts.

1.2 Points de Collecte

On ajoute maintenant les tables `CollectionPoint(id, adresse, capaciteMax)` contenant la liste des points de collecte et `accepts(#pointid, #wastetypeid)` indiquant quel point collecte utiliser pour quel(s) déchet(s)

- GET `/points` : Liste tous les points de collecte.
- GET `/points/id` : Détails d'un point incluant la liste imbriquée des `WasteType` acceptés.
- PATCH `/points/id` : Modification partielle (ex : adresse).
- DELETE `/points/id/clear` : Vide tous les dépôts d'un point de collecte. le calcul du "taux de remplissage" (Partie 2) pour ce point doit retomber à 0%. C'est l'action typique effectuée par un agent de collecte après avoir vidé physiquement la borne.

1.3 Tests avec Bruno

- L'application doit isoler les requêtes SQL dans des **DAO** et la connexion via un objet externalisant la connexion (DS).
- Un test de chaque endpoint doit figurer dans une collection **BRUNO** (incluant les cas d'erreur).